

Experiment – 3

Name: Simranjeet Kaur

UID: 24BAI70048

Branch: CSE – AIML (CS221)

Section/Group: 24AIT_NTPP-2A

Semester: 5th

Date of Performance: 27/07/2026

Subject: Full Stack-II

Subject Code: 24CSP-337

EXPERIMENT – 3.1

1. **AIM:** To design and implement a secure authentication system using JWT for user login and session management.

2. **OBJECTIVE:**

1. To understand authentication mechanisms in web applications.
2. To implement token-based authentication using JWT.
3. To manage user sessions in a stateless architecture.

3. **SOFTWARE REQUIREMENTS:**

1. Windows/Linux/macOS
2. Node.js (LTS Version)
3. NPM (Node Package Manager)
4. Visual Studio Code
5. Google Chrome / Microsoft Edge
6. React.js
7. Command Prompt / Terminal
8. JWT libraries

4. **THEORY**

Authentication is the process of verifying the identity of a user before allowing access to an application. In a web application, a user provides credentials such as a username and password. After successful verification, the application creates an authenticated session for the user.

JSON Web Token (JWT) is a token-based authentication mechanism used to securely transfer user information between the client and server. JWT allows authentication to be implemented in a stateless manner, where the server does not need to maintain a separate session for every logged-in user.

A JWT consists of three parts:

1. Header – Contains information about the token type and signing algorithm.
2. Payload – Contains user-related claims such as user ID, username, role, and expiration time.
3. Signature – Verifies the integrity of the token and ensures that the token has not been altered.

5. **IMPLEMENTATION/PROCEDURE**

1. Create a React application using Vite and install the required dependencies.
2. Create a Login component containing username and password fields.
3. Validate the entered credentials using predefined/mock credentials.
4. After successful authentication, generate a mock JWT containing basic user information.
5. Store the generated token in the browser's localStorage.
6. Maintain the authentication state in the React application using useState.
7. Create a protected Dashboard component that is displayed only when the user is authenticated.
8. Retrieve the stored token using localStorage.getItem() to check whether the user is logged in.

9. Display the dashboard when a valid authentication token is present; otherwise, display the login page.
10. Implement a Logout button that removes the JWT token from localStorage and changes the authentication state to logged out.

6. CODE

```
import { useState } from "react";
import Login from "./Login";
import Dashboard from "./Dashboard";
import { isAuthenticated } from "./auth";
function App() {
  const [authenticated, setAuthenticated] = useState(
    isAuthenticated()
  );
  return (
    <div>
      {authenticated ? (
        <Dashboard
          onLogout={() => setAuthenticated(false)}
        />
      ) : (
        <Login
          onLogin={() => setAuthenticated(true)}
        />
      )}
    </div>
  );
}
export default App;
```

7. OUTPUT

The authentication system worked successfully. When the user entered the correct username and password, the application generated a JWT token and stored it in the browser. The user was then allowed to access the dashboard. On clicking **Logout**, the token was removed and the user returned to the login page.

Thus, the experiment successfully demonstrated JWT-based authentication, token storage, protected access, session management, and logout.

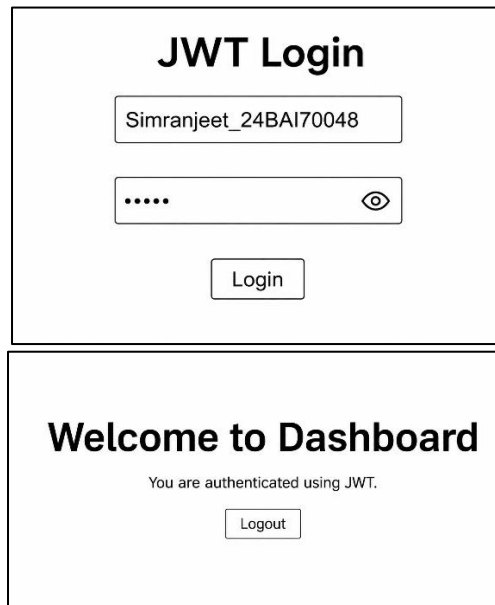


Fig 3.1: JWT Login and Authentication Dashboard



8. LEARNING OUTCOME

- Understand the concept of JWT-based authentication..
- Learn JWT token storage, validation, and expiration.
- Implement protected access and logout functionality.
- Implement user login and token-based session management
- Understand the stateless authentication flow in web applications.
- Understand the structure and components of a JSON Web Token (JWT).

EXPERIMENT –3.2

1. **AIM:** To implement role-based access control and secure application routes based on user permissions.

2. **OBJECTIVE:**

1. To understand authorization mechanisms in applications
2. To implement RBAC for access control
3. To protect routes based on user roles

3. **SOFTWARE REQUIREMENTS:**

1. Windows/Linux/macOS
2. Node.js (LTS Version)
3. npm (Node Package Manager)
4. Visual Studio Code
5. Google Chrome / Microsoft Edge
6. React.js
7. Command Prompt / Terminal
8. React Router

4. **THEORY**

Role-Based Access Control (RBAC) is an authorization mechanism used to control access to application features and resources based on the role assigned to a user. Authentication verifies the identity of a user, whereas authorization determines what that user is allowed to access. In this React application, three roles are defined: **Student, Staff, and Admission**. A Student can access academic features such as attendance, marks, and courses, while Staff can manage student attendance, marks, and records. The Admission role can manage admission applications, approvals, and student registration. The user's role is stored in the authentication state using the Context API, and React Router is used to manage application routes. Protected routes check whether the logged-in user's role has permission to access a particular page. If the user is authorized, the requested page is displayed; otherwise, access is denied and the user is redirected. The interface also dynamically displays features according to the user's role. Thus, RBAC helps provide secure navigation, controlled access, and a role-specific user interface in the application.

5. **IMPLEMENTATION/PROCEDURE**

1. Create a React application and install **React Router DOM** for route management.
2. Define three user roles: **Student, Staff, and Admission**.
3. Implement **Context API** to store and manage the logged-in user's role.
4. Create separate login options and dashboards for each user role.
5. Implement **protected routes** using React Router to restrict access based on user roles.
6. Use **conditional rendering** to display role-specific buttons, services, and dashboard features
7. Redirect unauthorized users when they try to access a restricted route and provide a logout option.
8. Run and test the application to verify that each role can access only its permitted features.

6. **CODE**

```
import { BrowserRouter, Routes, Route } from "react-router-dom";
import Login from "./pages/Login";
import Home from "./pages/Home";
import Student from "./pages/Student";
import Staff from "./pages/Staff";
import Admission from "./pages/Admission";
import ProtectedRoute from "./components/ProtectedRoute";
function App() {
  return (
    <BrowserRouter>
      <Routes>
```

```

    { /* Login */ }
    <Route path="/login" element={<Login />} />
    { /* Home */ }
    <Route
      path="/home"
      element={
        <ProtectedRoute
          allowedRoles={["Student", "Staff", "Admission"]}
        >
          <Home />
        </ProtectedRoute>
      }
    />
    { /* Student Route */ }
    <Route
      path="/student"
      element={
        <ProtectedRoute allowedRoles={["Student"]} >
          <Student />
        </ProtectedRoute>
      }
    />
    { /* Staff Route */ }
    <Route
      path="/staff"
      element={
        <ProtectedRoute allowedRoles={["Staff"]} >
          <Staff />
        </ProtectedRoute>
      }
    />
    { /* Admission Route */ }
    <Route
      path="/admission"
      element={
        <ProtectedRoute allowedRoles={["Admission"]} >
          <Admission />
        </ProtectedRoute>
      }
    />
    { /* Default */ }
    <Route path="*" element={<Login />} />

  </Routes>
</BrowserRouter>
);
}
export default App;

```

7. OUTPUT

The output shows a University Portal with Role-Based Access Control (RBAC). The login page provides three options: Student Login, Staff Login, and Admission Login. After selecting a role, the user is redirected to a role-specific dashboard. The Student dashboard provides options such as viewing attendance, marks, and courses; the Staff dashboard provides student, attendance, and marks management features; and the Admission dashboard provides application, approval, and registration features. The system checks the user's role before allowing access to protected routes. Thus, each user can access only the features authorized for their role, demonstrating secure navigation, protected routes, and dynamic role-based UI rendering.

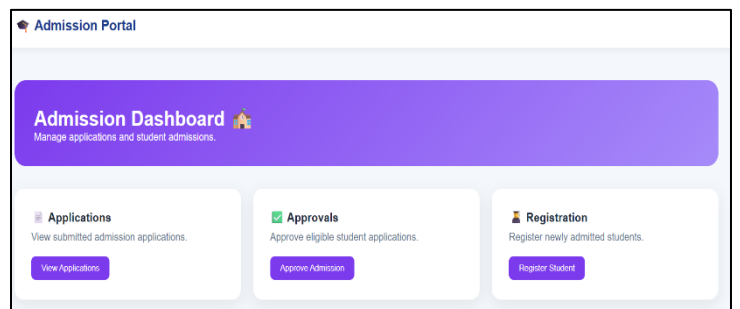
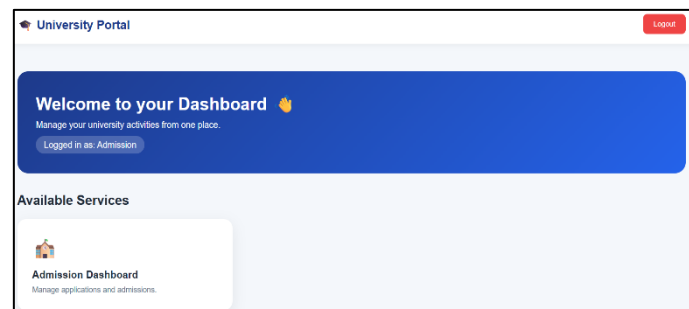
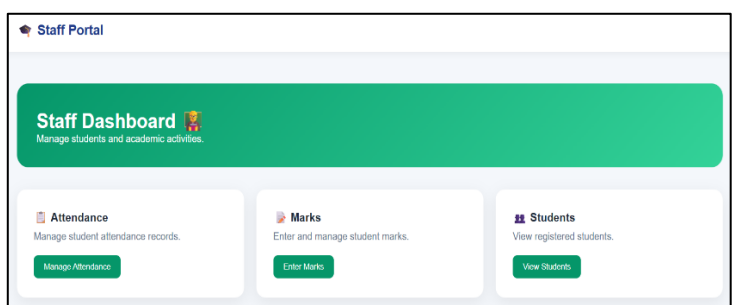
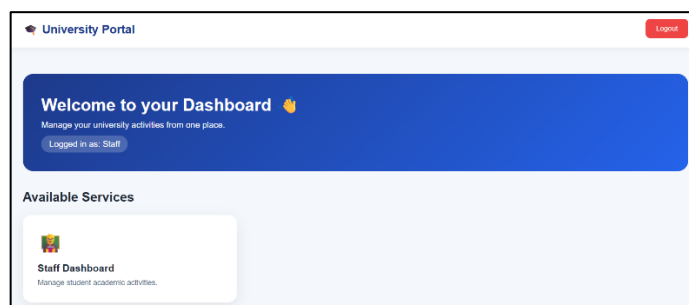
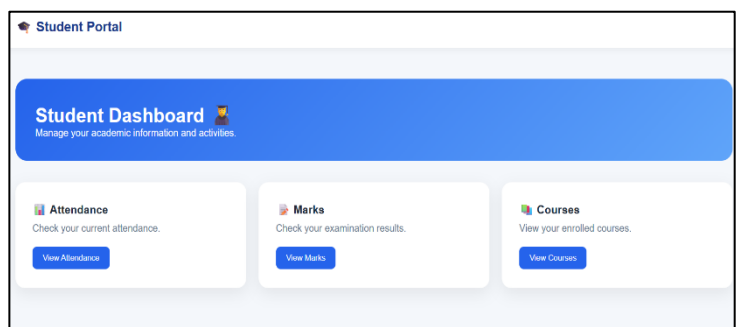
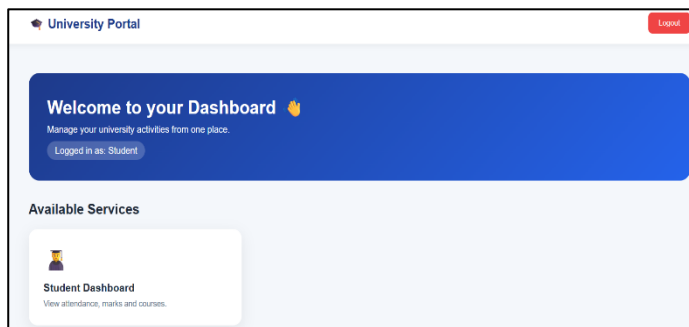
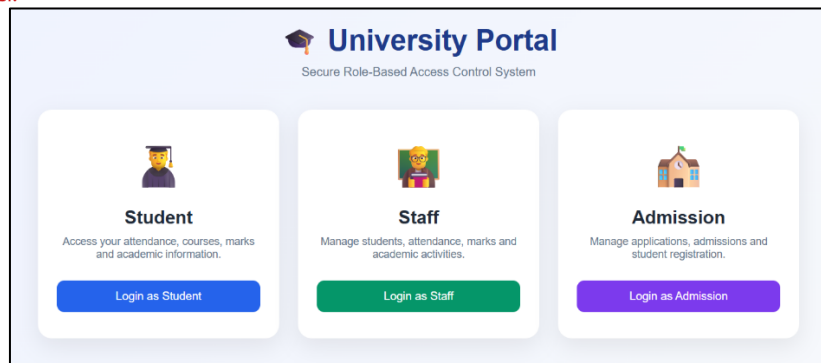


Fig 3.2: Role-Based Access Control (RBAC) showing role-specific dashboards for Student, Staff, and Admission users.

8. LEARNING OUTCOME

- Understand the concept of authorization and Role-Based Access Control (RBAC).
- Implement role-based authentication for Student, Staff, and Admission users.
- Create and manage protected routes using React Router.
- Restrict application features based on user permissions and roles.
- Implement dynamic UI rendering according to the logged-in user's role.
- Learn to redirect unauthorized users from restricted pages.